第④章

对象的组合

到目前为止，我们已经介绍了关于线程安全与同步的一些基础知识。然而，我们并不希望对每一次内存访问都进行分析以确保程序是线程安全的，而是希望将一些现有的线程安全组件组合为更大规模的组件或程序。本章将介绍一些组合模式，这些模式能够使一个类更容易成为线程安全的，并且在维护这些类时不会无意中破坏类的安全性保证。

4.1 设计线程安全的类

在线程安全的程序中，虽然可以将程序的所有状态都保存在公有的静态域中，但与那些将状态封装起来的程序相比，这些程序的线程安全性更难以得到验证，并且在修改时也更难以始终确保其线程安全性。通过使用封装技术，可以使得在不对整个程序进行分析的情况下就可以判断一个类是否是线程安全的。

在设计线程安全类的过程中，需要包含以下三个基本要素：

- 找出构成对象状态的所有变量。
- 找出约束状态变量的不变性条件。
- 建立对象状态的并发访问管理策略。

要分析对象的状态，首先从对象的域开始。如果对象中所有的域都是基本类型的变量，那么这些域将构成对象的全部状态。程序清单 4-1 中的 Counter 只有一个域 value，因此这个域就是 Counter 的全部状态。对于含有 n 个基本类型域的对象，其状态就是这些域构成的 n 元组。例如，二维点的状态就是它的坐标值 (x，y)。如果在对象的域中引用了其他对象，那么该对象的状态将包含被引用对象的域。例如，LinkedList 的状态就包括该链表中所有节点对象的状态。

程序清单 4-1　使用 Java 监视器模式的线程安全计数器

```
@ThreadSafe
public final class Counter {
    @GuardedBy("this")  private long value = 0;

    public synchronized  long getValue() {
        return value;
    }
    public  synchronized  long increment() {
        if (value == Long.MAX_VALUE)
```

```
            throw new IllegalStateException("counter overflow");
        return ++value;
    }
}
```

同步策略（Synchronization Policy）定义了如何在不违背对象不变条件或后验条件的情况下对其状态的访问操作进行协同。同步策略规定了如何将不可变性、线程封闭与加锁机制等结合起来以维护线程的安全性，并且还规定了哪些变量由哪些锁来保护。要确保开发人员可以对这个类进行分析与维护，就必须将同步策略写为正式文档。

4.1.1 收集同步需求

要确保类的线程安全性，就需要确保它的不变性条件不会在并发访问的情况下被破坏，这就需要对其状态进行推断。对象与变量都有一个状态空间，即所有可能的取值。状态空间越小，就越容易判断线程的状态。final 类型的域使用得越多，就越能简化对象可能状态的分析过程。（在极端的情况中，不可变对象只有唯一的状态。）

在许多类中都定义了一些不可变条件，用于判断状态是有效的还是无效的。Counter 中的 value 域是 long 类型的变量，其状态空间为从 Long.MIN_VALUE 到 Long.MAX_VALUE，但 Counter 中 value 在取值范围上存在着一个限制，即不能是负值。

同样，在操作中还会包含一些后验条件来判断状态迁移是否是有效的。如果 Counter 的当前状态为 17，那么下一个有效状态只能是 18。当下一个状态需要依赖当前状态时，这个操作就必须是一个复合操作。并非所有的操作都会在状态转换上施加限制，例如，当更新一个保存当前温度的变量时，该变量之前的状态并不会影响计算结果。

由于不变性条件以及后验条件在状态及状态转换上施加了各种约束，因此就需要额外的同步与封装。如果某些状态是无效的，那么必须对底层的状态变量进行封装，否则客户代码可能会使对象处于无效状态。如果在某个操作中存在无效的状态转换，那么该操作必须是原子的。另外，如果在类中没有施加这种约束，那么就可以放宽封装性或序列化等需求，以便获得更高的灵活性或性能。

在类中也可以包含同时约束多个状态变量的不变性条件。在一个表示数值范围的类（例如程序清单 4-10 中的 NumberRange）中可以包含两个状态变量，分别表示范围的上界和下界。这些变量必须遵循的约束是，下界值应该小于或等于上界值。类似于这种包含多个变量的不变性条件将带来原子性需求：这些相关的变量必须在单个原子操作中进行读取或更新。不能首先更新一个变量，然后释放锁并再次获得锁，然后再更新其他的变量。因为释放锁后，可能会使对象处于无效状态。如果在一个不变性条件中包含多个变量，那么在执行任何访问相关变量的操作时，都必须持有保护这些变量的锁。

如果不了解对象的不变性条件与后验条件，那么就不能确保线程安全性。要满足在状态变量的有效值或状态转换上的各种约束条件，就需要借助于原子性与封装性。

4.1.2 依赖状态的操作

类的不变性条件与后验条件约束了在对象上有哪些状态和状态转换是有效的。在某些对象的方法中还包含一些基于状态的先验条件（Precondition）。例如，不能从空队列中移除一个元素，在删除元素前，队列必须处于“非空的”状态。如果在某个操作中包含有基于状态的先验条件，那么这个操作就称为依赖状态的操作。

在单线程程序中，如果某个操作无法满足先验条件，那么就只能失败。但在并发程序中，先验条件可能会由于其他线程执行的操作而变成真。在并发程序中要一直等到先验条件为真，然后再执行该操作。

在 Java 中，等待某个条件为真的各种内置机制（包括等待和通知等机制）都与内置加锁机制紧密关联，要想正确地使用它们并不容易。要想实现某个等待先验条件为真时才执行的操作，一种更简单的方法是通过现有库中的类（例如阻塞队列 [Blocking Queue] 或信号量 [Semaphore]）来实现依赖状态的行为。第 5 章将介绍一些阻塞类，例如 BlockingQueue、Semaphore 以及其他的同步工具类。第 14 章将介绍如何使用在平台与类库中提供的各种底层机制来创建依赖状态的类。

4.1.3 状态的所有权

4.1 节曾指出，如果以某个对象为根节点构造一张对象图，那么该对象的状态将是对象图中所有对象包含的域的一个子集。为什么是一个“子集”？在从对象可以达到的所有域中，需要满足哪些条件才不属于对象状态的一部分？

在定义哪些变量将构成对象的状态时，只考虑对象拥有的数据。所有权（Ownership）在 Java 中并没有得到充分的体现，而是属于类设计中的一个要素。如果分配并填充了一个 HashMap 对象，那么就相当于创建了多个对象：HashMap 对象，在 HashMap 对象中包含的多个对象，以及在 Map.Entry 中可能包含的内部对象。HashMap 对象的逻辑状态包括所有的 Map.Entry 对象以及内部对象，即使这些对象都是一些独立的对象。

无论如何，垃圾回收机制使我们避免了如何处理所有权的问题。在 C++ 中，当把一个对象传递给某个方法时，必须认真考虑这种操作是否传递对象的所有权，是短期的所有权还是长期的所有权。在 Java 中同样存在这些所有权模型，只不过垃圾回收器为我们减少了许多在引用共享方面常见的错误，因此降低了在所有权处理上的开销。

许多情况下，所有权与封装性总是相互关联的：对象封装它拥有的状态，反之也成立，即对它封装的状态拥有所有权。状态变量的所有者将决定采用何种加锁协议来维持变量状态的完整性。所有权意味着控制权。然而，如果发布了某个可变对象的引用，那么就不再拥有独占的控制权，最多是“共享控制权”。对于从构造函数或者从方法中传递进来的对象，类通常并不拥有这些对象，除非这些方法是被专门设计为转移传递进来的对象的所有权（例如，同步容器封装器的工厂方法）。

容器类通常表现出一种“所有权分离”的形式，其中容器类拥有其自身的状态，而客户代码则拥有容器中各个对象的状态。Servlet 框架中的 ServletContext 就是其中一个示

例。ServletContext 为 Servlet 提供了类似于 Map 形式的对象容器服务，在 ServletContext 中可以通过名称来注册（setAttribute）或获取（getAttribute）应用程序对象。由 Servlet 容器实现的 ServletContext 对象必须是线程安全的，因为它肯定会被多个线程同时访问。当调用 setAttribute 和 getAttribute 时，Servlet 不需要使用同步，但当使用保存在 ServletContext 中的对象时，则可能需要使用同步。这些对象由应用程序拥有，Servlet 容器只是替应用程序保管它们。与所有共享对象一样，它们必须安全地被共享。为了防止多个线程在并发访问同一个对象时产生的相互干扰，这些对象应该要么是线程安全的对象，要么是事实不可变的对象，或者由锁来保护的对象。[⊖]

4.2 实例封闭

如果某对象不是线程安全的，那么可以通过多种技术使其在多线程程序中安全地使用。你可以确保该对象只能由单个线程访问（线程封闭），或者通过一个锁来保护对该对象的所有访问。

封装简化了线程安全类的实现过程，它提供了一种实例封闭机制（Instance Confinement），通常也简称为“封闭”[CPJ 2.3.3]。当一个对象被封装到另一个对象中时，能够访问被封装对象的所有代码路径都是已知的。与对象可以由整个程序访问的情况相比，更易于对代码进行分析。通过将封闭机制与合适的加锁策略结合起来，可以确保以线程安全的方式来使用非线程安全的对象。

将数据封装在对象内部，可以将数据的访问限制在对象的方法上，从而更容易确保线程在访问数据时总能持有正确的锁。

被封闭对象一定不能超出它们既定的作用域。对象可以封闭在类的一个实例（例如作为类的一个私有成员）中，或者封闭在某个作用域内（例如作为一个局部变量），再或者封闭在线程内（例如在某个线程中将对象从一个方法传递到另一个方法，而不是在多个线程之间共享该对象）。当然，对象本身不会逸出——出现逸出情况的原因通常是由于开发人员在发布对象时超出了对象既定的作用域。

程序清单 4-2 中的 PersonSet 说明了如何通过封闭与加锁等机制使一个类成为线程安全的（即使这个类的状态变量并不是线程安全的）。PersonSet 的状态由 HashSet 来管理的，而 HashSet 并非线程安全的。但由于 mySet 是私有的并且不会逸出，因此 HashSet 被封闭在 PersonSet 中。唯一能访问 mySet 的代码路径是 addPerson 与 containsPerson，在执行它们时都要获得 PersonSet 上的锁。PersonSet 的状态完全由它的内置锁保护，因而 PersonSet 是一个线

⊖ 需要注意的是，虽然 HttpSession 对象在功能上类似于 Servlet 框架，但可能有着更严格的要求。由于 Servlet 容器可能需要访问 HttpSession 中的对象，以便在复制操作或者钝化操作（Passivation，指的是将状态保存到持久性存储）中对它们序列化，因此这些对象必须是线程安全的，因为容器可能与 Web Application 程序同时访问它们。（之所以说“可能”，是因为在 Servlet 的规范中并没有明确定义复制与钝化等操作，这只是大多数 Servlet 容器的一个常见功能。）

程安全的类。

程序清单 4-2 通过封闭机制来确保线程安全

```
@ThreadSafe
public class PersonSet {
    @GuardedBy("this")
    private final Set<Person> mySet = new HashSet<Person>();

    public synchronized void addPerson(Person p) {
        mySet.add(p);
    }

    public synchronized boolean containsPerson(Person p) {
        return mySet.contains(p);
    }
}
```

这个示例并未对 Person 的线程安全性做任何假设，但如果 Person 类是可变的，那么在访问从 PersonSet 中获得的 Person 对象时，还需要额外的同步。要想安全地使用 Person 对象，最可靠的方法就是使 Person 成为一个线程安全的类。另外，也可以使用锁来保护 Person 对象，并确保所有客户代码在访问 Person 对象之前都已经获得正确的锁。

实例封闭是构建线程安全类的一个最简单方式，它还使得在锁策略的选择上拥有了更多的灵活性。在 PersonSet 中使用了它的内置锁来保护它的状态，但对于其他形式的锁来说，只要自始至终都使用同一个锁，就可以保护状态。实例封闭还使得不同的状态变量可以由不同的锁来保护。（后面章节的 ServerStatus 中就使用了多个锁来保护类的状态。）

在 Java 平台的类库中还有很多线程封闭的示例，其中有些类的唯一用途就是将非线程安全的类转化为线程安全的类。一些基本的容器类并非线程安全的，例如 ArrayList 和 HashMap，但类库提供了包装器工厂方法（例如 Collections.synchronizedList 及其类似方法），使得这些非线程安全的类可以在多线程环境中安全地使用。这些工厂方法通过“装饰器（Decorator）”模式 (Gamma et al., 1995) 将容器类封装在一个同步的包装器对象中，而包装器能将接口中的每个方法都实现为同步方法，并将调用请求转发到底层的容器对象上。只要包装器对象拥有对底层容器对象的唯一引用（即把底层容器对象封闭在包装器中），那么它就是线程安全的。在这些方法的 Javadoc 中指出，对底层容器对象的所有访问必须通过包装器来进行。

当然，如果将一个本该被封闭的对象发布出去，那么也能破坏封闭性。如果一个对象本应该封闭在特定的作用域内，那么让该对象逸出作用域就是一个错误。当发布其他对象时，例如迭代器或内部的类实例，可能会间接地发布被封闭对象，同样会使被封闭对象逸出。

封闭机制更易于构造线程安全的类，因为当封闭类的状态时，在分析类的线程安全性时就无须检查整个程序。

4.2.1 Java 监视器模式

从线程封闭原则及其逻辑推论可以得出 Java 监视器模式[⊖]。遵循 Java 监视器模式的对象会把对象的所有可变状态都封装起来，并由对象自己的内置锁来保护。

在程序清单 4-1 的 Counter 中给出了这种模式的一个典型示例。在 Counter 中封装了一个状态变量 value，对该变量的所有访问都需要通过 Counter 的方法来执行，并且这些方法都是同步的。

在许多类中都使用了 Java 监视器模式，例如 Vector 和 Hashtable。在某些情况下，程序需要一种更复杂的同步策略。第 11 章将介绍如何通过细粒度的加锁策略来提高可伸缩性。Java 监视器模式的主要优势就在于它的简单性。

Java 监视器模式仅仅是一种编写代码的约定，对于任何一种锁对象，只要自始至终都使用该锁对象，都可以用来保护对象的状态。程序清单 4-3 给出了如何使用私有锁来保护状态。

程序清单 4-3 通过一个私有锁来保护状态

```
public class PrivateLock {
    private final Object myLock = new Object();
    @GuardedBy("myLock") Widget widget;

    void someMethod() {
        synchronized(myLock) {
            // 访问或修改 Widget 的状态
        }
    }
}
```

使用私有的锁对象而不是对象的内置锁（或任何其他可通过公有方式访问的锁），有许多优点。私有的锁对象可以将锁封装起来，使客户代码无法得到锁，但客户代码可以通过公有方法来访问锁，以便（正确或者不正确地）参与到它的同步策略中。如果客户代码错误地获得了另一个对象的锁，那么可能会产生活跃性问题。此外，要想验证某个公有访问的锁在程序中是否被正确地使用，则需要检查整个程序，而不是单个的类。

4.2.2 示例：车辆追踪

程序清单 4-1 中的 Counter 是一个简单但用处不大的 Java 监视器模式示例。我们来看一个更有用处的示例：一个用于调度车辆的“车辆追踪器”，例如出租车、警车、货车等。首先使用监视器模式来构建车辆追踪器，然后再尝试放宽某些封装性需求同时又保持线程安全性。

每台车都由一个 String 对象来标识，并且拥有一个相应的位置坐标（x，y）。在 VehicleTracker 类中封装了车辆的标识和位置，因而它非常适合作为基于 MVC (Model-View-Controller，模型－视图－控制器) 模式的 GUI 应用程序中的数据模型，并且该模型将由一个视图线程和多

⊖ 虽然 Java 监视器模式来自于 Hoare 对监视器机制的研究工作（Hoare, 1974），但这种模式与真正的监视器类之间存在一些重要的差异。进入和退出同步代码块的字节指令也称为 monitorenter 和 monitorexit，而 Java 的内置锁也称为监视器锁或监视器。

个执行更新操作的线程共享。视图线程会读取车辆的名字和位置，并将它们显示在界面上：

```
Map<String, Point> locations = vehicles.getLocations();
for (String key : locations.keySet())
    renderVehicle(key, locations.get(key));
```

类似地，执行更新操作的线程通过从 GPS 设备上获取的数据或者调度员从 GUI 界面上输入的数据来修改车辆的位置。

```
void vehicleMoved(VehicleMovedEvent evt) {
    Point loc = evt.getNewLocation();
    vehicles.setLocation(evt.getVehicleId(), loc.x, loc.y);
}
```

视图线程与执行更新操作的线程将并发地访问数据模型，因此该模型必须是线程安全的。程序清单 4-4 给出了一个基于 Java 监视器模式实现的“车辆追踪器”，其中使用了程序清单 4-5 中的 MutablePoint 来表示车辆的位置。

程序清单 4-4 基于监视器模式的车辆追踪

```
@ThreadSafe
public class MonitorVehicleTracker {
    @GuardedBy("this")
    private final Map<String, MutablePoint> locations;

    public MonitorVehicleTracker(
            Map<String, MutablePoint> locations) {
        this.locations = deepCopy(locations);
    }

    public synchronized Map<String, MutablePoint> getLocations() {
        return deepCopy(locations);
    }

    public synchronized  MutablePoint getLocation(String id) {
        MutablePoint loc = locations.get(id);
        return loc == null ? null : new MutablePoint(loc);
    }

    public synchronized  void setLocation(String id, int x, int y) {
        MutablePoint loc = locations.get(id);
        if (loc == null)
            throw new IllegalArgumentException("No such ID: " + id);
        loc.x = x;
        loc.y = y;
    }

    private static Map<String, MutablePoint> deepCopy(
            Map<String, MutablePoint> m) {
        Map<String, MutablePoint> result =
                new HashMap<String, MutablePoint>();
        for (String id : m.keySet())
            result.put(id, new MutablePoint(m.get(id)));
        return Collections.unmodifiableMap(result);
    }
```

```
}

public class MutablePoint { /*  程序清单 4-5  */ }
```

虽然类 MutablePoint 不是线程安全的，但追踪器类是线程安全的。它所包含的 Map 对象和可变的 Point 对象都未曾发布。当需要返回车辆的位置时，通过 MutablePoint 拷贝构造函数或者 deepCopy 方法来复制正确的值，从而生成一个新的 Map 对象，并且该对象中的值与原有 Map 对象中的 key 值和 value 值都相同。[⊖]

程序清单 4-5 与 Java.awt.Point 类似的可变 Point 类（不要这么做）

```
@NotThreadSafe
public class MutablePoint {
    public int x, y;

    public MutablePoint() { x = 0; y = 0; }
    public MutablePoint(MutablePoint p) {
        this.x = p.x;
        this.y = p.y;
    }
}
```

在某种程度上，这种实现方式是通过在返回客户代码之前复制可变的数据来维持线程安全性的。通常情况下，这并不存在性能问题，但在车辆容器非常大的情况下将极大地降低性能[⊖]。此外，由于每次调用 getLocation 就要复制数据，因此将出现一种错误情况——虽然车辆的实际位置发生了变化，但返回的信息却保持不变。这种情况是好还是坏，要取决于你的需求。如果在 location 集合上存在内部的一致性需求，那么这就是优点，在这种情况下返回一致的快照就非常重要。然而，如果调用者需要每辆车的最新信息，那么这就是缺点，因为这需要非常频繁地刷新快照。

4.3 线程安全性的委托

大多数对象都是组合对象。当从头开始构建一个类，或者将多个非线程安全的类组合为一个类时，Java 监视器模式是非常有用的。但是，如果类中的各个组件都已经是线程安全的，会是什么情况呢？我们是否需要再增加一个额外的线程安全层？答案是“视情况而定”。在某些情况下，通过多个线程安全类组合而成的类是线程安全的（如程序清单 4-7 和程序清单 4-9 所示），而在某些情况下，这仅仅是一个好的开端（如程序清单 4-10 所示）。

⊖ 注意，deepCopy 并不只是用 unmodifiableMap 来包装 Map 的，因为这只能防止容器对象被修改，而不能防止调用者修改保存在容器中的可变对象。基于同样的原因，如果只是通过拷贝构造函数来填充 deepCopy 中的 HashMap，那么同样是不正确的，因为这样做只复制了指向 Point 对象的引用，而不是 Point 对象本身。

⊖ 由于 deepCopy 是从一个 synchronized 方法中调用的，因此在执行时间较长的复制操作中，tracker 的内置锁将一直被占有，当有大量车辆需要追踪时，会严重降低用户界面的响应灵敏度。

在前面的CountingFactorizer类中，我们在一个无状态的类中增加了一个AtomicLong类型的域，并且得到的组合对象仍然是线程安全的。由于CountingFactorizer的状态就是AtomicLong的状态，而AtomicLong是线程安全的，因此CountingFactorizer不会对counter的状态施加额外的有效性约束，所以很容易知道CountingFactorizer是线程安全的。我们可以说CountingFactorizer将它的线程安全性委托给AtomicLong来保证：之所以CountingFactorizer是线程安全的，是因为AtomicLong是线程安全的。[⊖]

4.3.1 示例：基于委托的车辆追踪器

下面将介绍一个更实际的委托示例，构造一个委托给线程安全类的车辆追踪器。我们将车辆的位置保存到一个Map对象中，因此首先要实现一个线程安全的Map类，ConcurrentHashMap。我们还可以用一个不可变的Point类来代替MutablePoint以保存位置，如程序清单4-6所示。

程序清单4-6 在DelegatingVehicleTracker中使用的不可变Point类

```
@Immutable
public class Point {
    public final int x, y;

    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }
}
```

由于Point类是不可变的，因而它是线程安全的。不可变的值可以被自由地共享与发布，因此在返回location时不需要复制。

在程序清单4-7的DelegatingVehicleTracker中没有使用任何显式的同步，所有对状态的访问都由ConcurrentHashMap来管理，而且Map所有的键和值都是不可变的。

程序清单4-7 将线程安全委托给ConcurrentHashMap

```
@ThreadSafe
public class DelegatingVehicleTracker {
    private final ConcurrentMap<String, Point> locations;
    private final Map<String, Point> unmodifiableMap;

    public DelegatingVehicleTracker(Map<String, Point> points) {
        locations = new ConcurrentHashMap<String, Point>(points);
        unmodifiableMap = Collections.unmodifiableMap(locations);
    }
```

⊖ 如果count不是final类型，那么要分析CountingFactorizer的线程安全性将变得更复杂。如果CountingFactorizer将count修改为指向另一个AtomicLong域的引用，那么必须确保count的更新操作对于所有访问count的线程都是可见的，并且还要确保在count的值上不存在竞态条件。这也是尽可能使用final类型域的另一个原因。

```
    public Map<String, Point> getLocations() {
        return unmodifiableMap;
    }

    public Point getLocation(String id) {
        return locations.get(id);
    }

    public void setLocation(String id, int x, int y) {
        if (locations.replace(id, new Point(x, y)) == null)
            throw new IllegalArgumentException(
                "invalid vehicle name: " + id);
    }
}
```

如果使用最初的 MutablePoint 类而不是 Point 类，就会破坏封装性，因为 getLocations 会发布一个指向可变状态的引用，而这个引用不是线程安全的。需要注意的是，我们稍微改变了车辆追踪器类的行为。在使用监视器模式的车辆追踪器中返回的是车辆位置的快照，而在使用委托的车辆追踪器中返回的是一个不可修改但却实时的车辆位置视图。这意味着，如果线程 A 调用 getLocations，而线程 B 在随后修改了某些点的位置，那么在返回给线程 A 的 Map 中将反映出这些变化。在前面提到过，这可能是一种优点（更新的数据），也可能是一种缺点（可能导致不一致的车辆位置视图），具体情况取决于你的需求。

如果需要一个不发生变化的车辆视图，那么 getLocations 可以返回对 locations 这个 Map 对象的一个浅拷贝 (Shallow Copy)。由于 Map 的内容是不可变的，因此只需复制 Map 的结构，而不用复制它的内容，如程序清单 4-8 所示（其中只返回一个 HashMap，因为 getLocations 并不能保证返回一个线程安全的 Map）。

程序清单 4-8 返回 locations 的静态拷贝而非实时拷贝

```
public Map<String, Point> getLocations() {
    return Collections.unmodifiableMap(
            new HashMap<String, Point>(locations));
}
```

4.3.2 独立的状态变量

到目前为止，这些委托示例都仅仅委托给了单个线程安全的状态变量。我们还可以将线程安全性委托给多个状态变量，只要这些变量是彼此独立的，即组合而成的类并不会在其包含的多个状态变量上增加任何不变性条件。

程序清单 4-9 中的 VisualComponent 是一个图形组件，允许客户程序注册监控鼠标和键盘等事件的监听器。它为每种类型的事件都备有一个已注册监听器列表，因此当某个事件发生时，就会调用相应的监听器。然而，在鼠标事件监听器与键盘事件监听器之间不存在任何关联，二者是彼此独立的，因此 VisualComponent 可以将其线程安全性委托给这两个线程安全的监听器列表。

程序清单 4-9 将线程安全性委托给多个状态变量

```
public class VisualComponent {
    private final List<KeyListener> keyListeners
        = new CopyOnWriteArrayList<KeyListener>();
    private final List<MouseListener> mouseListeners
        = new CopyOnWriteArrayList<MouseListener>();

    public void addKeyListener(KeyListener listener) {
        keyListeners.add(listener);
    }

    public void addMouseListener(MouseListener listener) {
        mouseListeners.add(listener);
    }

    public void removeKeyListener(KeyListener listener) {
        keyListeners.remove(listener);
    }

    public void removeMouseListener(MouseListener listener) {
        mouseListeners.remove(listener);
    }
}
```

VisualComponent 使用 CopyOnWriteArrayList 来保存各个监听器列表。它是一个线程安全的链表，特别适用于管理监听器列表（参见 5.2.3 节）。每个链表都是线程安全的，此外，由于各个状态之间不存在耦合关系，因此 VisualComponent 可以将它的线程安全性委托给 mouseListeners 和 keyListeners 等对象。

4.3.3 当委托失效时

大多数组合对象都不会像 VisualComponent 这样简单：在它们的状态变量之间存在着某些不变性条件。程序清单 4-10 中的 NumberRange 使用了两个 AtomicInteger 来管理状态，并且含有一个约束条件，即第一个数值要小于或等于第二个数值。

程序清单 4-10 NumberRange 类并不足以保护它的不变性条件（不要这么做）

```
public class NumberRange {
    //不变性条件：lower <= upper
    private final AtomicInteger lower = new AtomicInteger(0);
    private final AtomicInteger upper = new AtomicInteger(0);

    public void setLower(int i) {
    // 注意——不安全的“先检查后执行”
        if (i > upper.get())
            throw new IllegalArgumentException(
                    "can't set lower to " + i + " > upper");
        lower.set(i);
    }

    public void setUpper(int i) {
```

```
        // 注意 —— 不安全的“先检查后执行”
            if (i < lower.get())
                throw new IllegalArgumentException(
                        "can't set upper to " + i + " < lower");
            upper.set(i);
        }

        public boolean isInRange(int i) {
            return (i >= lower.get() && i <= upper.get());
        }
    }
```

NumberRange 不是线程安全的，没有维持对下界和上界进行约束的不变性条件。setLower 和 setUpper 等方法都尝试维持不变性条件，但却无法做到。setLower 和 setUpper 都是“先检查后执行”的操作，但它们没有使用足够的加锁机制来保证这些操作的原子性。假设取值范围为 (0，10)，如果一个线程调用 setLower(5)，而另一个线程调用 setUpper(4)，那么在一些错误的执行时序中，这两个调用都将通过检查，并且都能设置成功。结果得到的取值范围就是 (5，4)，那么这是一个无效的状态。因此，虽然 AtomicInteger 是线程安全的，但经过组合得到的类却不是。由于状态变量 lower 和 upper 不是彼此独立的，因此 NumberRange 不能将线程安全性委托给它的线程安全状态变量。

NumberRange 可以通过加锁机制来维护不变性条件以确保其线程安全性，例如使用一个锁来保护 lower 和 upper。此外，它还必须避免发布 lower 和 upper，从而防止客户代码破坏其不变性条件。

如果某个类含有复合操作，例如 NumberRange，那么仅靠委托并不足以实现线程安全性。在这种情况下，这个类必须提供自己的加锁机制以保证这些复合操作都是原子操作，除非整个复合操作都可以委托给状态变量。

如果一个类是由多个独立且线程安全的状态变量组成，并且在所有的操作中都不包含无效状态转换，那么可以将线程安全性委托给底层的状态变量。

即使 NumberRange 的各个状态组成部分都是线程安全的，也不能确保 NumberRange 的线程安全性，这种问题非常类似于 3.1.4 节介绍的 volatile 变量规则：仅当一个变量参与到包含其他状态变量的不变性条件时，才可以声明为 volatile 类型。

4.3.4 发布底层的状态变量

当把线程安全性委托给某个对象的底层状态变量时，在什么条件下才可以发布这些变量从而使其他类能修改它们？答案仍然取决于在类中对这些变量施加了哪些不变性条件。虽然 Counter 中的 value 域可以为任意整数值，但 Counter 施加的约束条件是只能取正整数，此外递增操作同样约束了下一个状态的有效取值范围。如果将 value 声明为一个公有域，那么客户代码可以将它修改为一个无效值，因此发布 value 会导致这个类出错。另一方面，如果某个变量

表示的是当前温度或者最近登录用户的 ID，那么即使另一个类在某个时刻修改了这个值，也不会破坏任何不变性条件，因此发布这个变量也是可以接受的。（这或许不是个好主意，因为发布可变的变量将对下一步的开发和派生子类带来限制，但不会破坏类的线程安全性。）

> 如果一个状态变量是线程安全的，并且没有任何不变性条件来约束它的值，在变量的操作上也不存在任何不允许的状态转换，那么就可以安全地发布这个变量。

例如，发布 VisualComponent 中的 mouseListeners 或 keyListeners 等变量就是安全的。由于 VisualComponent 并没有在其监听器链表的合法状态上施加任何约束，因此这些域可以声明为公有域或者发布，而不会破坏线程安全性。

4.3.5 示例：发布状态的车辆追踪器

我们来构造车辆追踪器的另一个版本，并在这个版本中发布底层的可变状态。我们需要修改接口以适应这种变化，即使用可变且线程安全的 Point 类。

程序清单 4-11 中的 SafePoint 提供的 get 方法同时获得 x 和 y 的值，并将二者放在一个数组中返回[⊖]。如果为 x 和 y 分别提供 get 方法，那么在获得这两个不同坐标的操作之间，x 和 y 的值发生变化，从而导致调用者看到不一致的值：车辆从来没有到达过位置（x，y）。通过使用 SafePoint，可以构造一个发布其底层可变状态的车辆追踪器，还能确保其线程安全性不被破坏，如程序清单 4-12 中的 PublishingVehicleTracker 类所示。

程序清单 4-11 线程安全且可变的 Point 类

```
@ThreadSafe
public class SafePoint {
    @GuardedBy("this") private int x, y;

    private SafePoint(int[] a) { this(a[0], a[1]); }

    public SafePoint(SafePoint p) { this(p.get()); }

    public SafePoint(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public synchronized int[] get() {
        return new int[] { x, y };
    }

    public synchronized void set(int x, int y) {
        this.x = x;
        this.y = y;
    }
}
```

⊖ 如果将拷贝构造函数实现为 this (p.x，p.y)，那么会产生竞态条件，而私有构造函数则可以避免这种竞态条件。这是私有构造函数捕获模式（Private Constructor Capture Idiom，Bloch and Gafter，2005）的一个实例.

程序清单 4-12 安全发布底层状态的车辆追踪器

```
@ThreadSafe
public class PublishingVehicleTracker {
    private final Map<String, SafePoint> locations;
    private final Map<String, SafePoint> unmodifiableMap;

    public PublishingVehicleTracker(
                        Map<String, SafePoint> locations) {
        this.locations
            = new ConcurrentHashMap<String, SafePoint>(locations);
        this.unmodifiableMap
            = Collections.unmodifiableMap(this.locations);
    }

    public Map<String, SafePoint> getLocations() {
        return unmodifiableMap;
    }

    public SafePoint getLocation(String id) {
        return locations.get(id);
    }

    public void setLocation(String id, int x, int y) {
        if (!locations.containsKey(id))
            throw new IllegalArgumentException(
                "invalid vehicle name: " + id);
        locations.get(id).set(x, y);
      }
}
```

PublishingVehicleTracker 将其线程安全性委托给底层的 ConcurrentHashMap，只是 Map 中的元素是线程安全的且可变的 Point，而并非不可变的。getLocation 方法返回底层 Map 对象的一个不可变副本。调用者不能增加或删除车辆，但却可以通过修改返回 Map 中的 SafePoint 值来改变车辆的位置。再次指出，Map 的这种“实时”特性究竟是带来好处还是坏处，仍然取决于实际的需求。PublishingVehicleTracker 是线程安全的，但如果它在车辆位置的有效值上施加了任何约束，那么就不再是线程安全的。如果需要对车辆位置的变化进行判断或者当位置变化时执行一些操作，那么 PublishingVehicleTracker 中采用的方法并不合适。

4.4 在现有的线程安全类中添加功能

Java 类库包含许多有用的“基础模块”类。通常，我们应该优先选择重用这些现有的类而不是创建新的类：重用能降低开发工作量、开发风险（因为现有的类都已经通过测试）以及维护成本。有时候，某个现有的线程安全类能支持我们需要的所有操作，但更多时候，现有的类只能支持大部分的操作，此时就需要在不破坏线程安全性的情况下添加一个新的操作。

例如，假设需要一个线程安全的链表，它需要提供一个原子的“若没有则添加（Put-If-Absent）”的操作。同步的 List 类已经实现了大部分的功能，我们可以根据它提供的 contains 方法和 add 方法来构造一个“若没有则添加”的操作。

“若没有则添加”的概念很简单，在向容器中添加元素前，首先检查该元素是否已经存在，如果存在就不再添加。（回想“先检查再执行”的注意事项。）由于这个类必须是线程安全的，因此就隐含地增加了另一个需求，即“若没有则添加”这个操作必须是原子操作。这意味着，如果在链表中没有包含对象 X，那么在执行两次“若没有则添加”X 后，在容器中只能包含一个 X 对象。然而，如果“若没有则添加”操作不是原子操作，那么在某些执行情况下，有两个线程都将看到 X 不在容器中，并且都执行了添加 X 的操作，从而使容器中包含两个相同的 X 对象。

要添加一个新的原子操作，最安全的方法是修改原始的类，但这通常无法做到，因为你可能无法访问或修改类的源代码。要想修改原始的类，就需要理解代码中的同步策略，这样增加的功能才能与原有的设计保持一致。如果直接将新方法添加到类中，那么意味着实现同步策略的所有代码仍然处于一个源代码文件中，从而更容易理解与维护。

另一种方法是扩展这个类，假定在设计这个类时考虑了可扩展性。程序清单 4-13 中的 BetterVector 对 Vector 进行了扩展，并添加了一个新方法 putIfAbsent。扩展 Vector 很简单，但并非所有的类都像 Vector 那样将状态向子类公开，因此也就不适合采用这种方法。

程序清单 4-13 扩展 Vector 并增加一个“若没有则添加”方法

```
@ThreadSafe
public class BetterVector<E> extends Vector<E> {
    public synchronized boolean putIfAbsent(E x) {
        boolean absent = !contains(x);
        if (absent)
            add(x);
        return absent;
    }
}
```

“扩展”方法比直接将代码添加到类中更加脆弱，因为现在的同步策略实现被分布到多个单独维护的源代码文件中。如果底层的类改变了同步策略并选择了不同的锁来保护它的状态变量，那么子类会被破坏，因为在同步策略改变后它无法再使用正确的锁来控制对基类状态的并发访问。（在 Vector 的规范中定义了它的同步策略，因此 BetterVector 不存在这个问题。）

4.4.1 客户端加锁机制

对于由 Collections.synchronizedList 封装的 ArrayList，这两种方法在原始类中添加一个方法或者对类进行扩展都行不通，因为客户代码并不知道在同步封装器工厂方法中返回的 List 对象的类型。第三种策略是扩展类的功能，但并不是扩展类本身，而是将扩展代码放入一个“辅助类”中。

程序清单 4-14 实现了一个包含“若没有则添加”操作的辅助类，用于对线程安全的 List 执

行操作，但其中的代码是错误的。

程序清单 4-14 非线程安全的“若没有则添加”（不要这么做）

```
@NotThreadSafe
public class ListHelper<E> {
    public List<E> list =
        Collections.synchronizedList(new ArrayList<E>());
    ...
    public synchronized boolean putIfAbsent(E x) {
        boolean absent = !list.contains(x);
        if (absent)
            list.add(x);
        return absent;
    }
}
```

为什么这种方式不能实现线程安全性？毕竟，putIfAbsent 已经声明为 synchronized 类型的变量，对不对？问题在于在错误的锁上进行了同步。无论 List 使用哪一个锁来保护它的状态，可以确定的是，这个锁并不是 ListHelper 上的锁。ListHelper 只是带来了同步的假象，尽管所有的链表操作都被声明为 synchronized，但却使用了不同的锁，这意味着 putIfAbsent 相对于 List 的其他操作来说并不是原子的，因此就无法确保当 putIfAbsent 执行时另一个线程不会修改链表。

要想使这个方法能正确执行，必须使 List 在实现客户端加锁或外部加锁时使用同一个锁。客户端加锁是指，对于使用某个对象 X 的客户端代码，使用 X 本身用于保护其状态的锁来保护这段客户代码。要使用客户端加锁，你必须知道对象 X 使用的是哪一个锁。

在 Vector 和同步封装器类的文档中指出，它们通过使用 Vector 或封装器容器的内置锁来支持客户端加锁。程序清单 4-15 给出了在线程安全的 List 上执行 putIfAbsent 操作，其中使用了正确的客户端加锁。

程序清单 4-15 通过客户端加锁来实现“若没有则添加”

```
@ThreadSafe
public class ListHelper<E> {
    public List<E> list =
        Collections.synchronizedList(new ArrayList<E>());
    ...
    public boolean putIfAbsent(E x) {
        synchronized (list)  {
            boolean absent = !list.contains(x);
            if (absent)
                list.add(x);
            return absent;
        }
    }
}
```

通过添加一个原子操作来扩展类是脆弱的，因为它将类的加锁代码分布到多个类中。然

而，客户端加锁却更加脆弱，因为它将类 C 的加锁代码放到与 C 完全无关的其他类中。当在那些并不承诺遵循加锁策略的类上使用客户端加锁时，要特别小心。

客户端加锁机制与扩展类机制有许多共同点，二者都是将派生类的行为与基类的实现耦合在一起。正如扩展会破坏实现的封装性 [EJ Item 14]，客户端加锁同样会破坏同步策略的封装性。

4.4.2 组合

当为现有的类添加一个原子操作时，有一种更好的方法：组合（Composition）。程序清单 4-16 中的 ImprovedList 通过将 List 对象的操作委托给底层的 List 实例来实现 List 的操作，同时还添加了一个原子的 putIfAbsent 方法。（与 Collections.synchronizedList 和其他容器封装器一样，ImprovedList 假设把某个链表对象传给构造函数以后，客户代码不会再直接使用这个对象，而只能通过 ImprovedList 来访问它。）

程序清单 4-16 通过组合实现“若没有则添加”

```
@ThreadSafe
public class ImprovedList<T> implements List<T> {
    private final List<T> list;

    public ImprovedList(List<T> list) { this.list = list; }

    public synchronized boolean putIfAbsent(T x) {
        boolean contains = list.contains(x);
        if (contains)
            list.add(x);
        return !contains;
    }

    public synchronized void clear() { list.clear(); }
    // ... 按照类似的方式委托 List 的其他方法
}
```

ImprovedList 通过自身的内置锁增加了一层额外的加锁。它并不关心底层的 List 是否是线程安全的，即使 List 不是线程安全的或者修改了它的加锁实现，ImprovedList 也会提供一致的加锁机制来实现线程安全性。虽然额外的同步层可能导致轻微的性能损失[⊖]，但与模拟另一个对象的加锁策略相比，ImprovedList 更为健壮。事实上，我们使用了 Java 监视器模式来封装现有的 List，并且只要在类中拥有指向底层 List 的唯一外部引用，就能确保线程安全性。

4.5 将同步策略文档化

在维护线程安全性时，文档是最强大的（同时也是最未被充分利用的）工具之一。用户可以通过查阅文档来判断某个类是否是线程安全的，而维护人员也可以通过查阅文档来理解其

⊖ 性能损失很小，因为在底层 List 上的同步不存在竞争，所以速度很快，请参见第 11 章。

中的实现策略，避免在维护过程中破坏安全性。然而，通常人们从文档中获取的信息却是少之又少。

在文档中说明客户代码需要了解的线程安全性保证，以及代码维护人员需要了解的同步策略。

synchronized、volatile 或者任何一个线程安全类都对应于某种同步策略，用于在并发访问时确保数据的完整性。这种策略是程序设计的要素之一，因此应该将其文档化。当然，设计阶段是编写设计决策文档的最佳时间。这之后的几周或几个月后，一些设计细节会逐渐变得模糊，因此一定要在忘记之前将它们记录下来。

在设计同步策略时需要考虑多个方面，例如，将哪些变量声明为 volatile 类型，哪些变量用锁来保护，哪些锁保护哪些变量，哪些变量必须是不可变的或者被封闭在线程中的，哪些操作必须是原子操作等。其中某些方面是严格的实现细节，应该将它们文档化以便于日后的维护。还有一些方面会影响类中加锁行为的外在表现，也应该将其作为规范的一部分写入文档。

最起码，应该保证将类中的线程安全性文档化。它是否是线程安全的？在执行回调时是否持有一个锁？是否有某些特定的锁会影响其行为？不要让客户冒着风险去猜测。如果你不想支持客户端加锁也是可以的，但一定要明确地指出来。如果你希望客户代码能够在类中添加新的原子操作，如 4.4 节所示，那么就需要在文档中说明需要获得哪些锁才能实现安全的原子操作。如果使用锁来保护状态，那么也要将其写入文档以便日后维护，这很简单，只需使用标注 @GuardedBy 即可。如果要使用更复杂的方法来维护线程安全性，那么一定要将它们写入文档，因为维护者通常很难发现它们。

甚至在平台的类库中，线程安全性方面的文档也是很难令人满意。当你阅读某个类的 Javadoc 时，是否曾怀疑过它是否是线程安全的？[⊖]大多数类都没有给出任何提示。许多正式的 Java 技术规范，例如 Servlet 和 JDBC，也没有在它们的文档中给出线程安全性的保证和需求。

尽管我们不应该对规范之外的行为进行猜测，但有时候出于工作需要，将不得不面对各种糟糕的假设。我们是否应该因为某个对象看上去是线程安全的而就假设它是安全的？是否可以假设通过获取对象的锁来确保对象访问的线程安全性？（只有当我们能控制所有访问该对象的代码时，才能使用这种带风险的技术，否则，这只能带来线程安全性的假象。）无论做出哪种选择都难以令人满意。

更糟糕的是，我们的直觉通常是错误的：我们认为“可能是线程安全“的类通常并不是线程安全的。例如，java.text.SimpleDateFormat 并不是线程安全的，但 JDK 1.4 之前的 Javadoc 并没有提到这点。许多开发人员都对这个类不是线程安全的而感到惊讶。有多少程序已经错误地生成了这种非线程安全的对象，并在多线程中使用它？这些程序没有意识到这将在高负载的情况下导致错误的结果。

如果某个类没有明确地声明是线程安全的，那么就不要假设它是线程安全的，从而有

⊖ 如果你从未考虑过这些问题，那么你确实比较乐观。

效地避免类似于 SimpleDateFormat 的问题。而另一方面，如果不对容器提供对象（例如 HttpSession）的线程安全性做某种有问题的假设，也就不可能开发出一个基于 Servlet 的应用程序。不要使你的客户或同事也做这样的猜测。

解释含糊的文档

许多 Java 技术规范都没有（或者至少不愿意）说明接口的线程安全性，例如 ServletContext、HttpSession 或 DataSource[⊖]。这些接口是由容器或数据库供应商来实现的，而你通常无法通过查看其实现代码来了解细节功能。此外，你也不希望依赖于某个特定 JDBC 驱动的实现细节——你希望遵从标准，这样代码可以基于任何一个 JDBC 驱动工作。但在 JDBC 的规范中从未出现“线程”和“并发”这些术语，同样在 Servlet 规范中也很少提到。那么你该做些什么呢？

你只能去猜测。一个提高猜测准确性的方法是，从实现者（例如容器或数据库的供应商）的角度去解释规范，而不是从使用者的角度去解释。Servlet 通常是在容器管理的（Container-Managed）线程中调用的，因此可以安全地假设：如果有多个这种线程在运行，那么容器是知道这种情况的。Servlet 容器能生成一些为多个 Servlet 提供服务的对象，例如 HttpSession 或 ServletContext。因此，Servlet 容器应该预见到这些对象将被并发访问，因为它创建了多个线程，并且从这些线程中调用像 Servlet.service 这样的方法，而这个方法很可能会访问 ServletContext。

由于这些对象在单线程的上下文中很少是有用的，因此我们不得不假设它们已被实现为线程安全的，即使在规范中没有明确地说明。此外，如果它们需要客户端加锁，那么客户端代码应该在哪个锁上进行同步？在文档中没有说明这一点，而要猜测的话也不知从何猜起。在规范和正式手册中给出的如何访问 ServletContext 或 HttpSession 的示例中进一步强调了这种“合理的假设”，并且没有使用任何客户端同步。

另一方面，通过把 setAttribute 放到 ServletContext 中或者将 HttpSession 的对象由 Web 应用程序拥有，而不是 Servlet 容器拥有。在 Serviet 规范中没有给出任何机制来协调对这些共享属性的并发访问。因此，由容器代替 Web 应用程序来保存这些属性应该是线程安全的，或者是不可变的。如果容器的工作只是代替 Web 应用程序来保存这些属性，那么当从 servlet 应用程序代码访问它们时，应该确保它们始终由同一个锁保护。但由于容器可能需要序列化 HttpSession 中的对象以实现复制或钝化等操作，并且容器不可能知道你的加锁协议，因此你要自己确保这些对象是线程安全的。

可以对 JDBC DataSource 接口做出类似的推断，该接口表示一个可重用的数据库连接池。DataSource 为应用程序提供服务，它在单线程应用程序中没有太大意义。我们很难想象不在多线程情况下使用 getConnection。并且，与 Servlet 一样，在使用 DataSource 的许多示例代码中，JDBC 规范并没有说明需要使用任何客户端加锁。因此，尽管 JDBC 规范没有说明 DataSource 是否是线程安全的，或者要求生产商提供线程安全的实现，但同样由于“如果不

⊖ 令我们失望的是，在多次对规范的修订中一直都忽略了这些问题。

这么做将是不可思议的”，所以我们只能假设 DataSource.getConnection 不需要额外的客户端加锁。

另一方面，在 DataSource 分配 JDBC Connection 对象上没有这样的争议，因为在它们返回连接池之前，不会有其他操作将它们共享。因此，如果某个获取 JDBC Connection 对象的操作跨越了多个线程，那么它必须通过同步来保护对 Connection 对象的访问。（大多数应用程序在实现使用 JDBC Connection 对象的操作时，通常都会把 Connection 对象封闭在某个特定的线程中。）